

Assignment 3

Responsive Web Design · Media Queries + Bootstrap Grid

Name: Miras Zhumazhan | Group: IT-2512

Source repository: <https://github.com/mmionya/web-technologies-assignment3>

Project and setup

This project is a fan-made Jamie Paige listening guide. The page includes an introduction, three reasons to listen, an album collection and a questions section. It uses local artwork and links to official music sources.

index.html connects Bootstrap 5.3.8 CSS from its CDN, followed by style.css. The viewport meta tag sets the layout width to the device width. The Bootstrap JavaScript bundle appears before the closing body tag and powers the menu and accordion.

Task 1 · A section with custom media queries

The #values section uses only custom classes. Its .values-grid starts with one column. At min-width: 768px it changes to two columns; at min-width: 1200px it changes to three. The media queries also adjust the heading size and spacing. This is mobile-first because the narrow layout works before either media query applies.

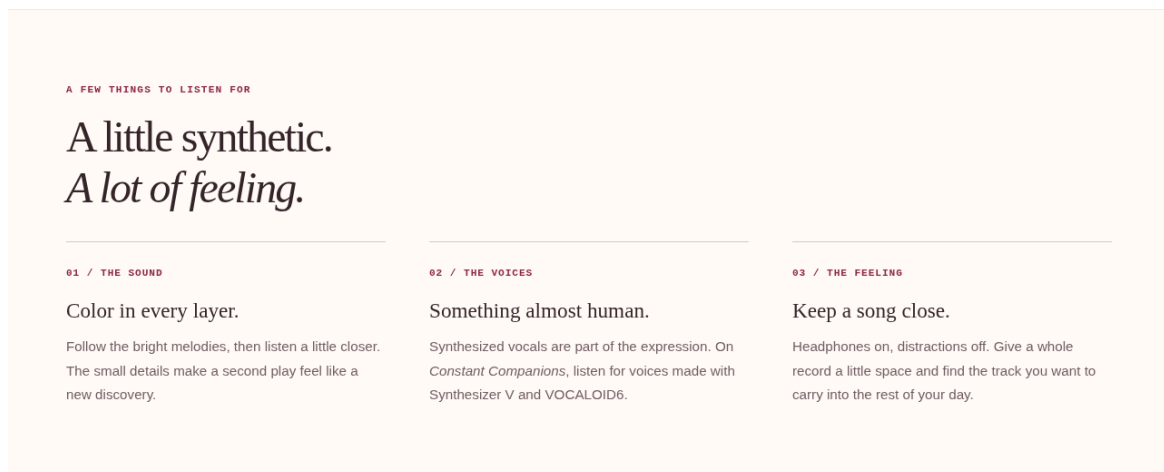


Figure 1. Three custom CSS columns at a 1280 px viewport.

Technical references: <https://getbootstrap.com/docs/5.3/getting-started/introduction/> and <https://getbootstrap.com/docs/5.3/layout/grid/>. Artwork credits and official album sources are recorded in assets/CREDITS.md; README.md includes the component documentation links.

Task 1 · Phone and tablet evidence

At 375 px, each reason occupies its own row. At 768 px, two reasons share the first row and the third continues below. CSS Grid handles the remaining space without fixed item widths or absolute positioning. The screenshots below show the same section at two viewport widths.

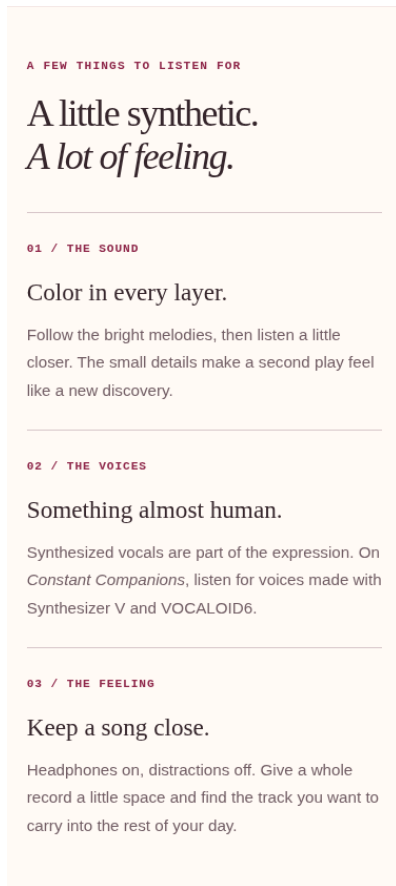


Figure 2. 375 px: one column.

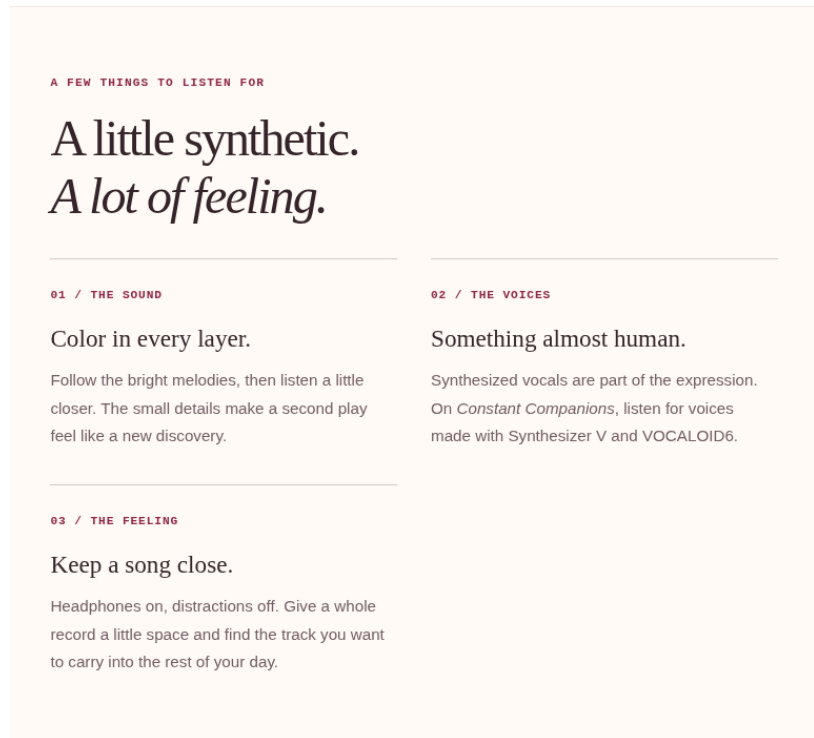


Figure 3. 768 px: two columns.

Task 2 · Bootstrap grid

The album section uses `.container`, `.row` and responsive columns: `.col-12 .col-md-6 .col-xl-4`. Bootstrap's 12-column grid therefore shows one album at 375 px, two at 768 px and three at 1280 px. Bootstrap cards keep the artwork, titles and links together; spacing utilities add consistent gaps.

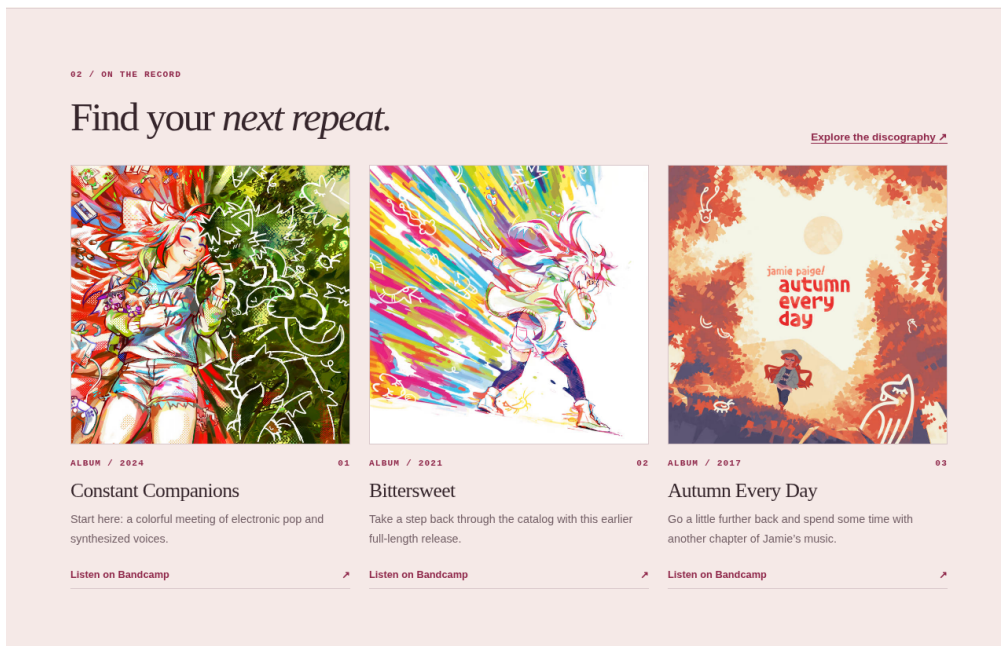


Figure 4. Three Bootstrap album cards at 1280 px.

The introduction uses two Bootstrap columns on wider screens. Responsive order-* classes put the introduction text before the artwork on a phone, so the purpose of the page appears first. On a desktop, the columns sit beside each other.

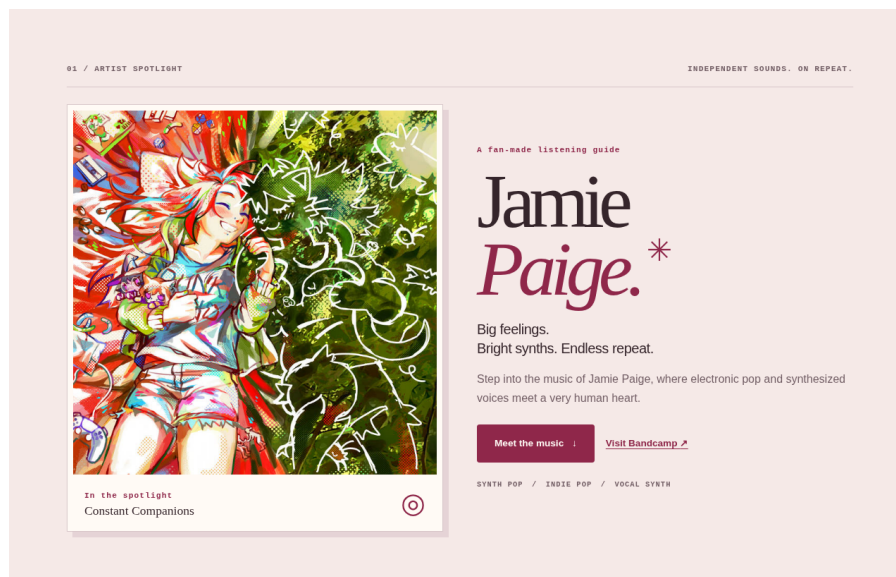


Figure 5. The introduction's two-column desktop layout.

Task 2 · Mobile order and interactive components

The .navbar-expand-lg navigation collapses below 992 px. Its menu button reveals links to the page sections. The introduction's slogan uses .d-none .d-md-inline, hiding below 768 px; the decorative FAQ flower uses .d-none .d-lg-block. The FAQ is a Bootstrap accordion: its buttons reveal answers through data-bs-* attributes, so the page does not need custom JavaScript.

These phone captures show the introduction with text first, the expanded menu, and an open answer. Browser checks also verify all three required viewport widths, both grids, section links, interactive controls and the absence of horizontal scrolling.

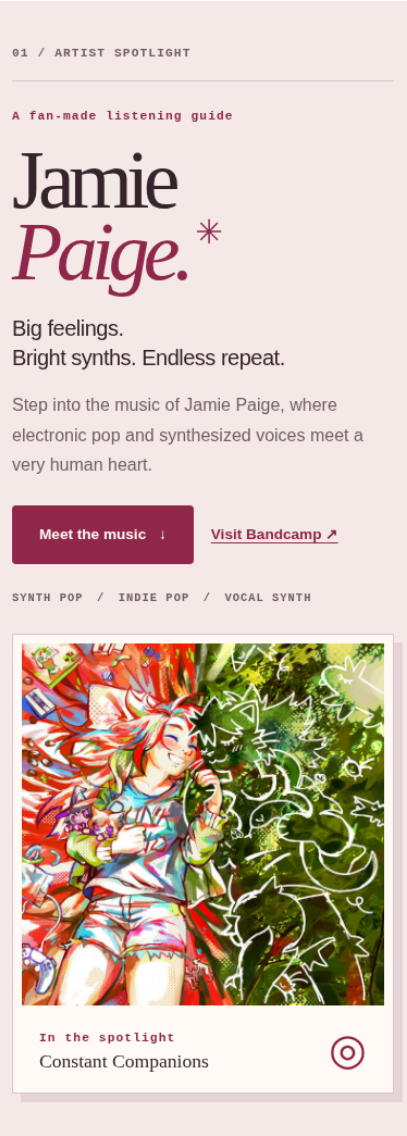


Figure 6. Text comes first at 375 px.

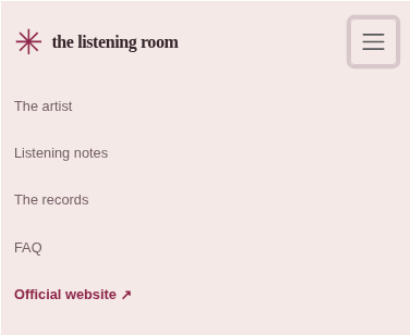


Figure 7. The expanded mobile navigation.

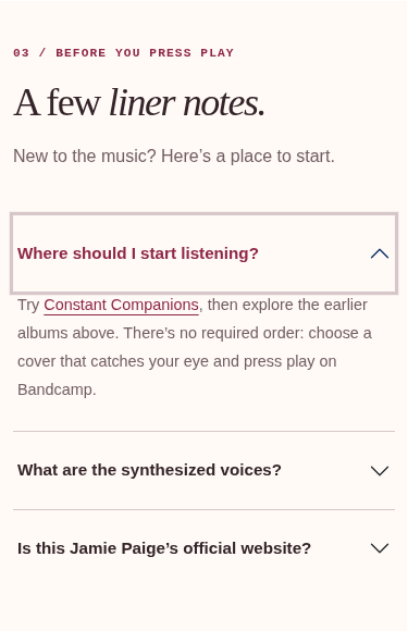
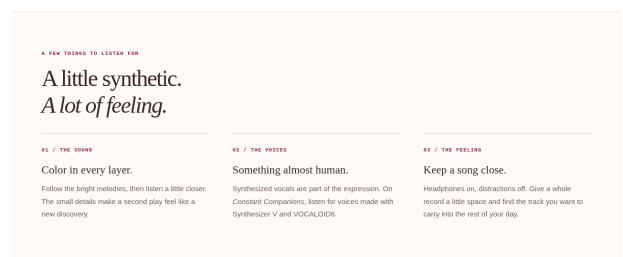
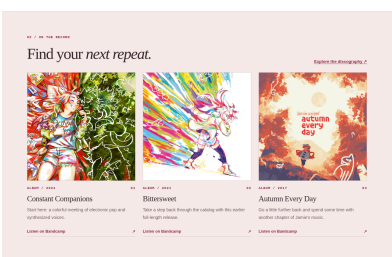


Figure 8. An expanded FAQ answer.

Task 3 · Comparison of the two approaches



Custom CSS at 1280 px (detail: Figure 1).



Bootstrap at 1280 px (detail: Figure 4).

Aspect	Custom CSS: reasons to listen	Bootstrap: albums and introduction
Responsive code	The .values-grid rule defines the base layout; two min-width media queries change its column count.	The album columns use col-12 col-md-6 col-xl-4. Bootstrap already defines the breakpoint rules.
Design control	The values section's heading size, gap and section spacing are adjusted directly in style.css.	The grid supplies column widths and gutters. Custom CSS supplies the pink palette, type and card appearance.
Changing the layout	To change the desktop count, edit grid-template-columns in the 1200 px media query.	To change the desktop count, edit each album's col-xl-* class; the CSS framework supplies the widths.
Built-in behavior	CSS Grid handles the repeating values, but does not provide an interactive menu or FAQ.	The Bootstrap bundle handles navbar collapse and accordion state through data attributes.

I would choose custom media queries for a small section that needs its own spacing and layout rules, such as the reasons-to-listen grid. I would choose Bootstrap when several page sections need consistent responsive columns and standard components, as with the albums, navigation and FAQ. Combining them works here because the custom section stays independent while the rest reuses Bootstrap.

Conclusion

This page shows how a mobile-first layout grows from one column to several columns as more space becomes available. The main difference is where the responsive rules live: in my own media queries or in Bootstrap's predefined classes. Testing at 375, 768 and 1280 px also makes it easier to catch ordering problems, overflow and controls that are difficult to use on a phone.

Validation: tests/check_page.py passed at 320, 375, 768, 1024, 1280 and 1440 px, including both grids, section links, menu and accordion keyboard controls, image loading and no horizontal overflow.

Whole-page screenshot · 375 px

Captured from the implemented page in a real browser. The entire page is included; use the task figures for close-up details.

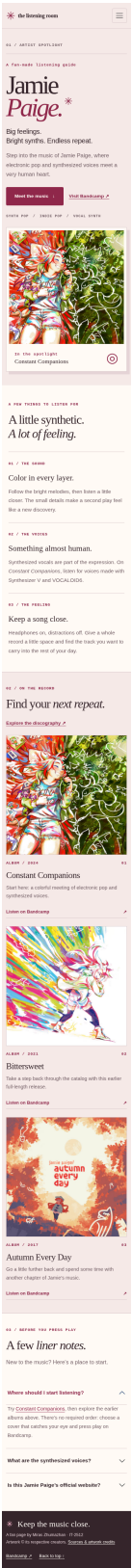


Figure 9. Complete page at a 375 px viewport.

Whole-page screenshot · 768 px

Captured from the implemented page in a real browser. The entire page is included; use the task figures for close-up details.

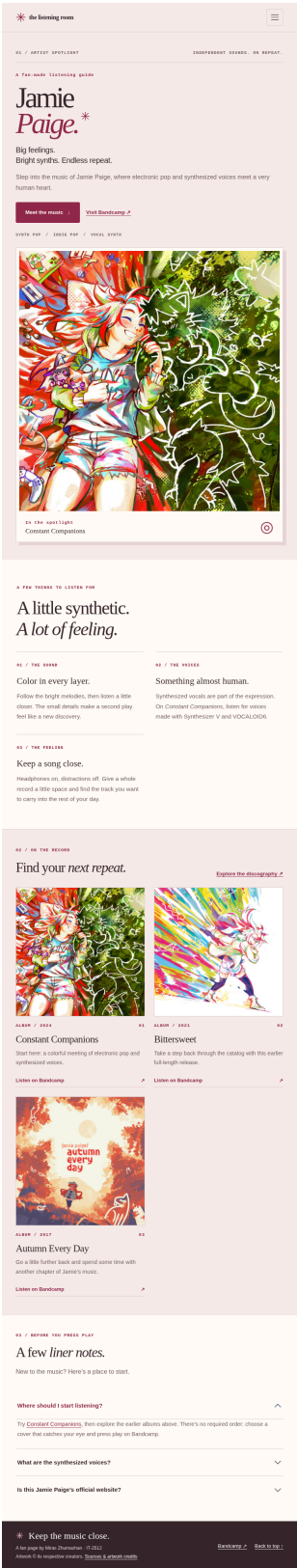


Figure 10. Complete page at a 768 px viewport.

Whole-page screenshot · 1280 px

Captured from the implemented page in a real browser. The entire page is included; use the task figures for close-up details.

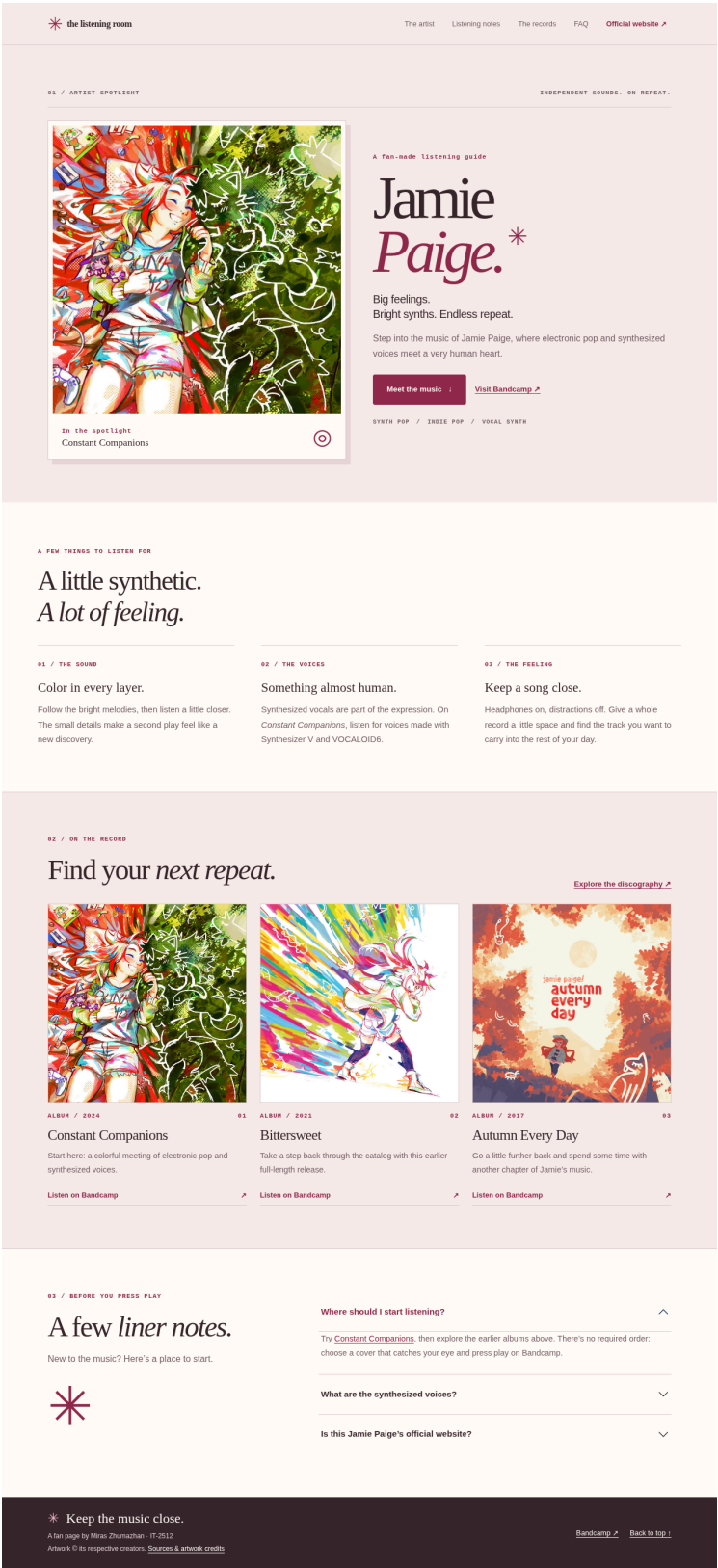


Figure 11. Complete page at a 1280 px viewport.